AI Engineering and Claude Architect · Part V, Certification

Certification, Lesson 3

Pydantic, MCP Resources and the Remaining Terms

The guide's appendix names a few technologies the Java track only touched on. This lesson fills those gaps with real, runnable validation and a clear view of what an MCP resource is for.

Module	AI13 of 13
Lesson	3 of 5
Prerequisites	AI03 L3, AI06 L4
Exam relevance	**Core: TS 4.3, 4.4, 2.4.** Also S6, S4
Reading time	~25 minutes

Contents

1. Pydantic, MCP Resources and the Remaining Terms

Lesson 3 of 5 · Certification

Pydantic, MCP Resources and the Remaining Terms

Key idea

A schema guarantees shape, while your validator checks meaning. Pydantic is how the guide's Python world expresses both, and a validation-retry loop feeds specific errors back, until retrying can't help. Separately, **MCP resources let an agent see what exists without making exploratory tool calls.**

> *Before: structured output and extraction (AI03 L3), MCP architecture (AI06 L4). Teaches: Pydantic field and model validators, a retry loop that knows when to stop, MCP resources in detail, and a term check against the guide's appendix. Used by: L4, L5.*

The hook: valid, and wrong

An invented example. An invoice extraction returns perfect JSON through a forced tool call. The line items are 40 and 50, and `calculated_total` is 80. Every field has the right type, so the schema is satisfied, and a wrong total reaches the ledger. The tool call eliminated syntax errors, as the guide says, and did nothing for this one. Something has to check meaning.

Intuition: a form and its reviewer

A web form makes sure a date is a date. It can't tell you that the delivery date comes before the order date. For that you need a reviewer who reads the whole form. Field validators are the form, and model validators are the reviewer.

Mechanism: two levels of validation

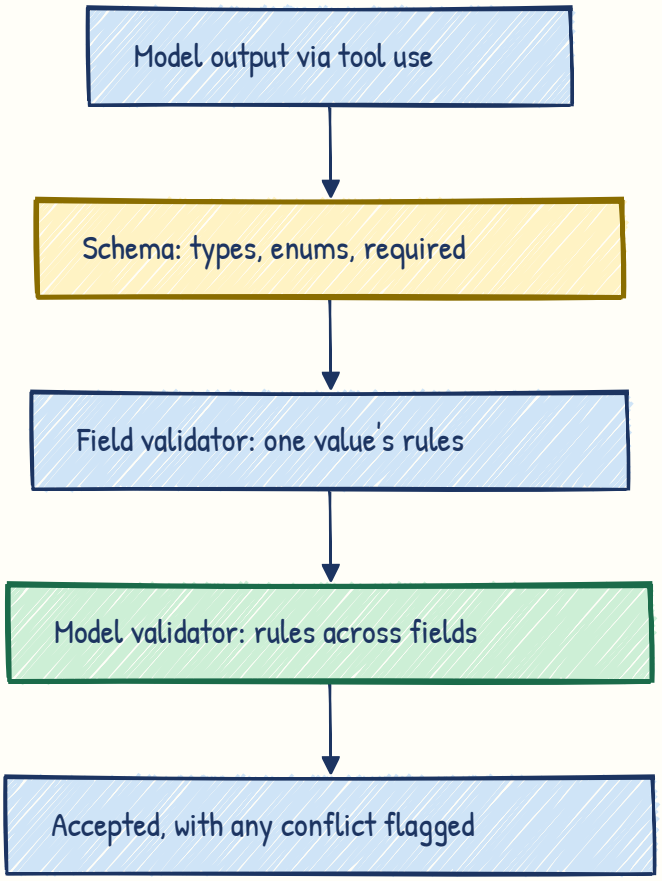


Figure 1. Three layers. Tool use removes the first kind of error, and the remaining kinds need your code.

This code comes from `code/ai13-exam/validation_lab.py`, run on Pydantic 2.14 with a scripted fake model. No real model is called, so it tests the loop's decisions, not extraction quality.

```
class Invoice(BaseModel):
    invoice_number: str
    category: Literal["goods", "services", "other", "unclear"]
    category_detail: Optional[str] = None
    purchase_order: Optional[str] = None    # nullable: many invoices have none
    line_items: list[LineItem]
    stated_total: Decimal
    calculated_total: Decimal

    @field_validator("invoice_number")
    @classmethod
    def looks_like_invoice(cls, value: str) -> str:
        if not value.startswith("INV-"):
            raise ValueError("invoice_number must start with INV-")
        return value

    @model_validator(mode="after")
    def totals_agree(self):
        total = sum(item.amount for item in self.line_items)
        if total != self.calculated_total:
            raise ValueError(f"calculated_total is {self.calculated_total}, "
                             f"but line_items sum to {total}")
        return self
```

Line	What decision it carries	Guide link
`Literal[..., "other", "unclear"]`	Escape values, so the model needn't force a fit	TS 4.3
`Optional[...] = None`	Absent means null, which removes pressure to invent	TS 4.3
`calculated_total` beside `stated_total`	One is computed and one is read, so a mismatch reveals something	TS 4.4
`@field_validator`	A rule about one value	Semantic error
`@model_validator(mode="after")`	A rule across fields, on the whole object	Semantic error
`raise ValueError`	Pydantic wraps it into a `ValidationError`, with a location and message	Feedback for a retry

Validators must return the value (or `self`), or the result is lost. That's a Pydantic rule from its docs, and a common bug.

The retry loop, and when it must stop

The guide's follow-up contains **the original document, the failed extraction and the specific errors.**

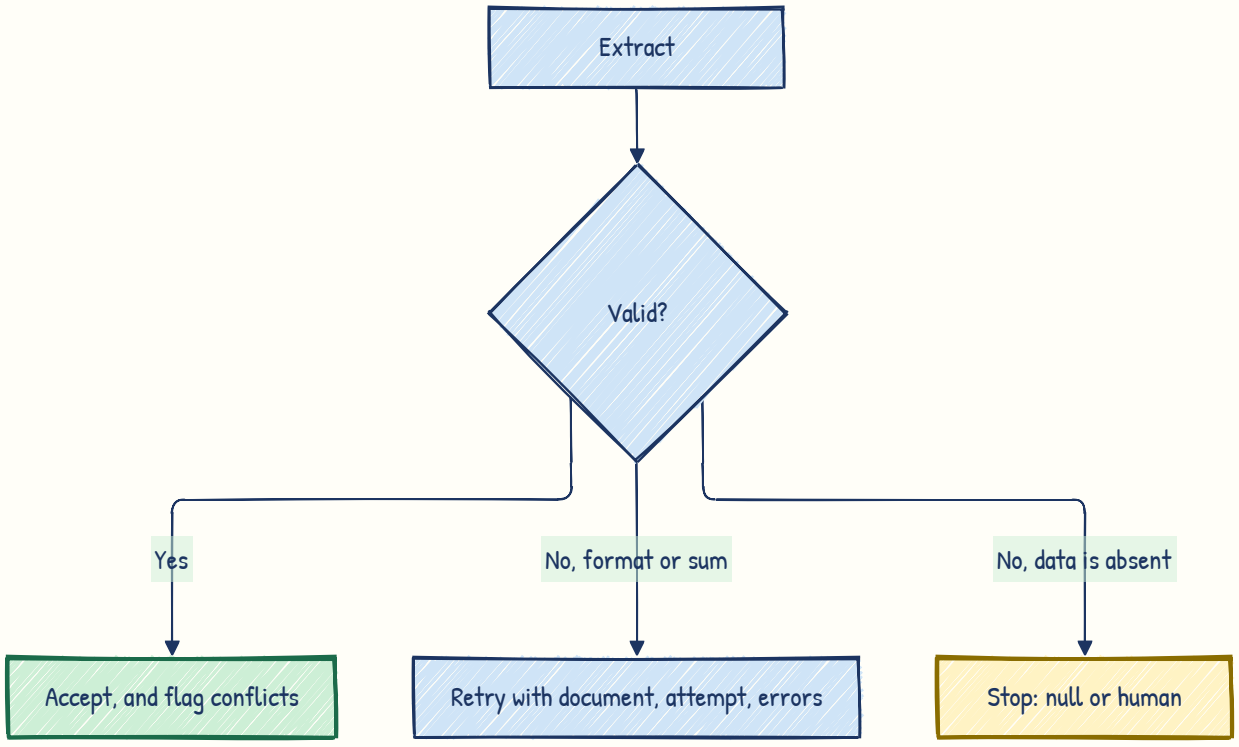


Figure 2. Validation retry. The loop's value is knowing which failures a retry can fix.

Real output:

```
A. Wrong sum, fixed by feedback
  attempt 1: invalid -> document: Value error, calculated_total is 80.00,
             but line_items sum to 90.00
  attempt 2: valid, conflict_detected=False
B. The document's own total disagrees: valid output, flagged, not 'fixed'
  attempt 1: valid, conflict_detected=True
C. The value isn't in the document: retrying repeats the error
  attempt 1: invalid -> invoice_number: Value error, invoice_number must start with INV-...
  attempt 2: invalid -> invoice_number: Value error, invoice_number must start with INV-...
  same error twice: the fix isn't in the document, so stop and route to a human
D. A schema error, seen without tool use
  category: Input should be 'goods', 'services', 'other' or 'unclear'
E. Nullable field: missing means None, not an invented value
  purchase_order = None
```

(Two long lines are wrapped or shortened here for the page.)

Case B matters most. `stated_total` is 95 and the items sum to 90. That isn't an extraction error, since the source really disagrees. Retrying would push the model to change what it read. The right result is valid output with `conflict_detected` set, and a human or downstream rule decides. **Case C** shows the guide's retry limit, and the same-error-twice stop is my illustration of detecting it. In real use, a nullable field would let the model return null there, instead of failing.

Failure	Will a retry help?	What to do
Format mismatch	Yes	Retry with the error
Sum or field placement error	Yes	Retry with the error
Value absent from the document	No	Null, or human review
Value is in another document not provided	No	Get that document first
Source contradicts itself	No	Flag the conflict

Why the obvious fixes fall short

Fix	Why it falls short	Better
"Just use tool use"	Removes syntax errors only	Add semantic validators
Generic retry: "try again"	No information to correct with	Send the specific errors
Make all fields required	Pressures the model to invent values	Nullable where data may be absent
Always retry 3 times	Wastes money on unfixable cases	Classify the failure first
Auto-correct the total in code	Hides a real discrepancy	Keep both totals, and flag it

How the field solved it

Pydantic documents field and model validators, and the wrapping of `ValueError` into `ValidationError`, so that's the source for the mechanism. Anthropic's documentation covers the structured output side, as AI03 L3 teaches. **Combining them into a retry loop follows the exam guide,** and the code is this course's own.

In Java, AI03 L3 uses records plus plain checks, and Jakarta Bean Validation is the nearest equivalent. Which library doesn't matter. The exam tests the layering.

MCP resources

AI06 L4 introduced the three primitives. Here's the depth the exam can reach, from the MCP specification.

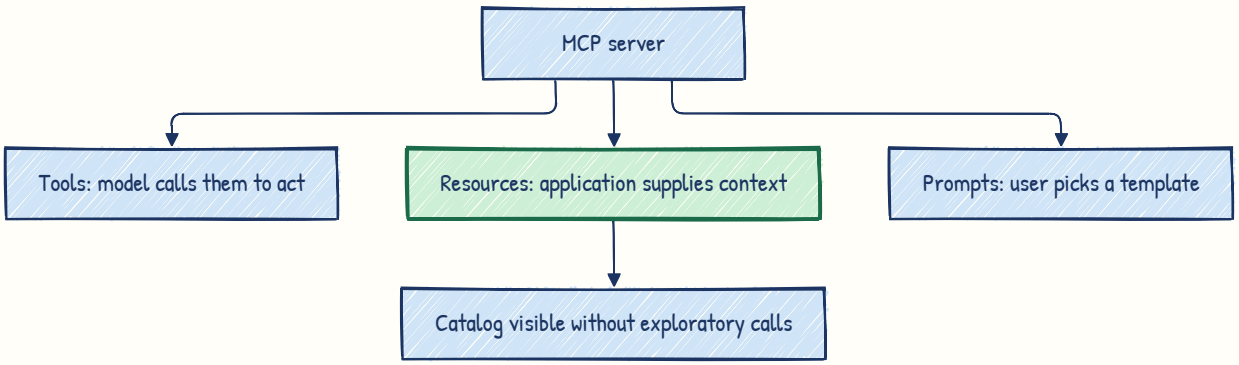


Figure 3. Who controls each MCP primitive. A resource gives the agent a map, and a tool takes an action.

Aspect	Resource	Tool
Purpose	Share data that gives context	Perform an operation
Control	Application-driven, per the spec	Model-invoked
Identified by	A URI	A name and schema
Discovery	`resources/list`, `resources/templates/list`	`tools/list`
Fetch or use	`resources/read`	`tools/call`
Updates	Optional `listChanged` and subscriptions	Optional list-changed notification

The exam's use (TS 2.4): expose a content catalog, such as issue summaries, a documentation hierarchy or database schemas, as a resource. The agent then knows what exists, and needn't make calls just to explore. The spec describes application-driven use, but also allows automatic inclusion "based on heuristics or the AI model's selection", so don't assume the model can never select a resource.

Example shape: a schema catalog at `db://reporting/schema`, with a parameterized template such as `db://reporting/tables/{name}` for per-table detail, and a `run_query` tool for actions. The URI scheme here is invented, which the spec allows. A resource that doesn't exist must return an error, not empty contents, for the same reason as TS 2.2: empty is ambiguous.

Term check against the guide's appendix

Everything the appendix names, and where it's taught:

Appendix item	Where
Agent SDK: definitions, loops, `stop_reason`, hooks, Task, `allowedTools`	AI06 L2, L6, L8, and L2 here
MCP: servers, tools, resources, `isError`, `.mcp.json`, env expansion	AI06 L4, L5, and this lesson
Claude Code: hierarchy, rules, commands, skills, plan mode, `/memory`, `/compact`, `--resume`, `fork_session`, Explore	AI12 L1–L4, AI06 L7
CLI: `-p`, `--output-format json`, `--json-schema`	AI12 L5, and L2 here
API: `tool_use`, `tool_choice`, `stop_reason`, `max_tokens`, system prompts	AI03 L1, L3
Message Batches: 50%, 24 hours, `custom_id`, polling, no multi-turn	AI03 L5
JSON Schema, and Pydantic	AI03 L3, and this lesson
Built-in tools	AI12 L1
Few-shot and prompt chaining	AI03 L2, AI06 L1
Context: token budgets, progressive summarization, lost in the middle, scratchpads	AI03 L6, AI06 L7
Sessions and confidence scoring	AI06 L7, AI10 L3

The full line-by-line result is in `01-source-map/EXAM_COVERAGE_MATRIX.md`.

The production architecture

For S6, here's the extraction pipeline with these pieces in place. **This is a recommended reference design, not a specific company's system.**

Baseline: a forced tool call, and accept the output. **New problem:** valid but wrong values. **Improved:** add validators and a retry loop. **New problem:** silent errors in documents that pass, and reviewers overloaded. **Production:** add confidence routing and stratified sampling, from AI10 L3.

Component	Why it exists	How it connects
Extraction tool	Guaranteed shape	Forced `tool_choice`
Pydantic model	Semantic checks	Validates the `tool_use` input
Retry controller	Fixes what can be fixed	Follow-up with errors
Review queue	Handles unfixable and low-confidence cases	Receives flagged items
Sampler	Catches errors in what passes	Samples high-confidence output

Failure	Result	Mitigation
Validator too strict	Good documents rejected	Measure rejects by type
Retry cost grows	Spend without gain	Stop on repeated errors, and cap attempts
Calibrated threshold drifts	Review misses new errors	Keep sampling

Java

The Java track already has this: see AI03 L3's extraction code, and AI08's tests. This lesson's lab is in Python because the appendix names Pydantic.

Trade-offs

Validation in code catches real errors, but it costs maintenance and can reject valid edge cases. **When not to use a retry loop:** when the error rate is so low, or a document so expensive, that a human route is cheaper. Measure before adding it.

Common mistakes

"Strict mode" or tool use doesn't validate meaning. If an item asks how to catch line items that don't sum to the total, an answer about schemas or `strict` is wrong.

Tip

Ask of every field: could the source lack this? If so, make it nullable. And of every failure: is the fix in the document? If not, a retry won't help.

Certification lens

Key idea

Supports **TS 4.3, 4.4, 2.4,** and the guide's appendix. Pydantic and MCP facts come from their own docs. The retry and stop logic is the guide's principle, in code that's my own, tested with canned replies only.

These are original practice questions, not real or leaked.

Q Q1 · Exam style, scenario: Structured Data Extraction · Medium. Output is valid JSON through a forced tool call, yet some invoices have line items that don't sum to `calculated_total`. What should you add?

A. Stricter JSON Schema types

B. Semantic validation in code, with a retry that sends the specific error

C. `tool_choice: "any"`

D. A lower temperature

A **Answer: B.** It's a semantic error, which schemas can't catch. Feeding back the specific error gives the model what it needs to correct itself.

Q Q2 · Exam style, scenario: Structured Data Extraction · Medium. A contract refers to an annex that wasn't supplied, and the renewal date is only in that annex. Validation fails on every retry. What's right?

A. Add more retries

B. Stop retrying, and get the annex or return null for review

C. Tell the model to estimate

D. Remove the field from the schema

A **Answer: B.** The information isn't in the source, so retries can't succeed. C invites fabrication, and D loses the requirement.

Q Q3 · Exam style, scenario: Developer Productivity with Claude · Medium. Agents make many exploratory calls just to learn which database tables exist. What best reduces them?

A. Expose a schema catalog as an MCP resource

B. Add a tool per table

C. Raise the turn limit

D. Paste all schemas into every system prompt

A **Answer: A.** The guide names this use. B adds tool-selection load, and D wastes context on every request.

Q Q4 · Exam style, scenario: Structured Data Extraction · Hard. `stated_total` reads 95, and the items sum to 90. The extraction read both correctly. What should the pipeline do?

A. Retry until they match

B. Replace the stated total with 90

C. Accept it, with `conflict_detected` true, for review

D. Reject the document as unreadable

A **Answer: C.** The source itself disagrees, so preserve both and flag it. A and B destroy evidence.

Revision sheet

→ Tool use removes syntax errors. Semantic errors need code.

→ `@field_validator` checks one value, and `@model_validator(mode="after")` checks across fields. Both must return.

→ A `ValueError` raised in a validator surfaces as a `ValidationError`.

→ A retry includes the document, the failed output and specific errors.

→ Retries fix format and structure, but not absent data.

→ Source disagreement: keep both values, and set a conflict flag.

→ Nullable fields and "unclear" or "other" enums reduce invention.

→ MCP resources: URI-identified context, listed and read, application-driven.

→ Use them to expose catalogs and reduce exploratory calls.

Your notes